Backtracking Algorithms

Backtracking is a general algorithmic technique used to solve problems incrementally by trying out all possible solutions. It explores the solution space recursively and abandons (backtracks) as soon as it determines a path cannot lead to a valid solution.

Core Idea:

- Choose → Try a candidate.
- Explore → If the candidate leads towards a solution, continue recursively.
- Un-choose → If the candidate leads to a dead end, undo choice and backtrack.

15 Puzzle Problem:

A sliding puzzle consisting of a 4×4 grid with 15 numbered tiles (1–15) and 1 empty slot.

Objective: Rearrange tiles to reach the goal state:

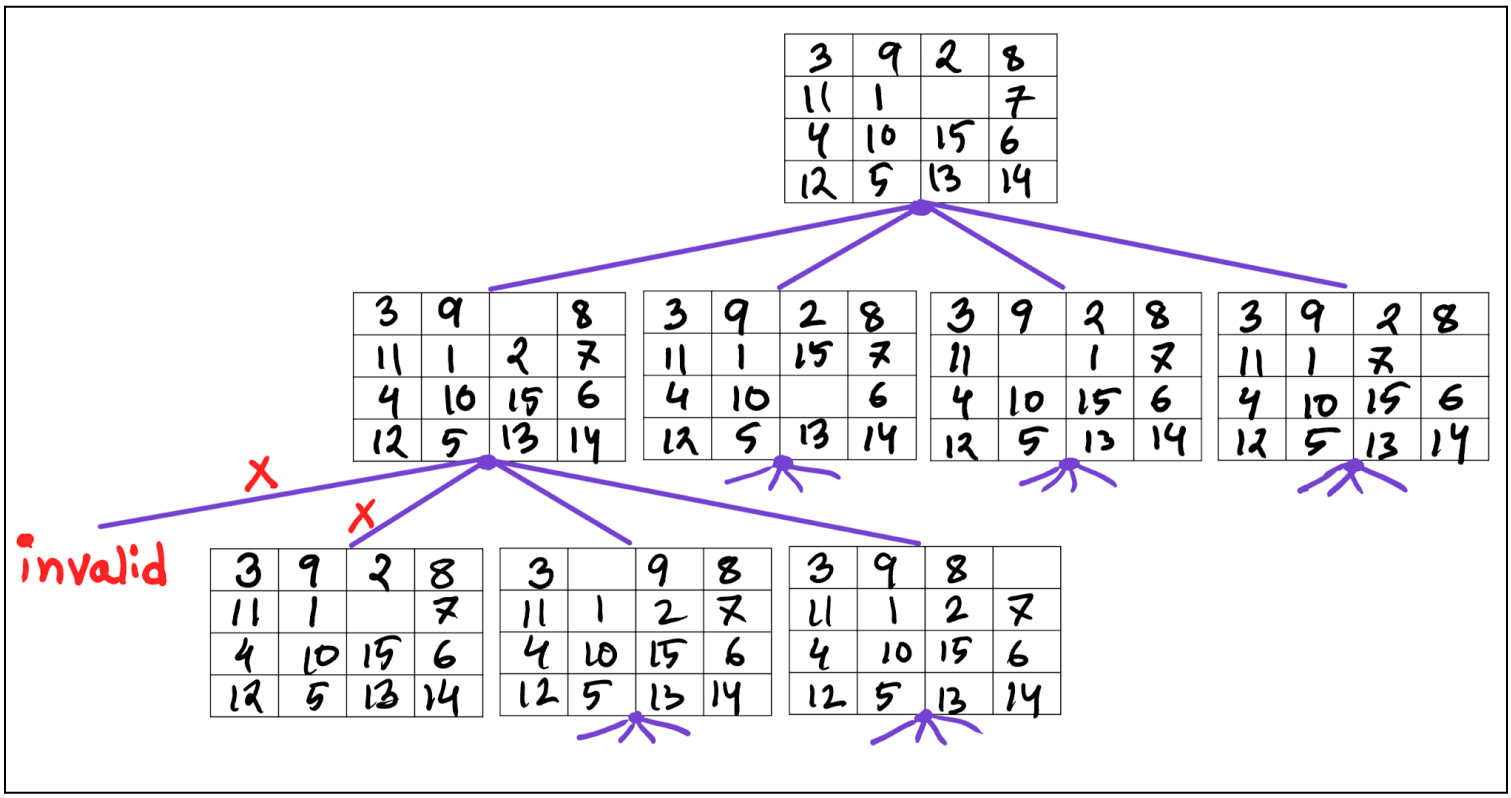
1	**2**	**3**	**4**
5	**6**	**7**	**8**
9	**10**	**11**	**12**
13	**14**	**15**	**–**

You may try out the game for better understanding: Link

Rules:

- A tile adjacent to the empty slot can slide into it.
- Only legal moves (up, down, left, right) are allowed.
- Problem: Find a sequence of moves that transforms the initial state into the goal state.

State Space Tree:



Pseudo Code:

```
function solve_puzzle(board, visited):

    if board == goal_state:
        return True   # Solution found

    mark board as visited

    for move in possible_moves(board):  # up, down, left, right
        if move is valid:
            new_board = apply_move(board, move)

        if new_board not in visited:  # To avoid getting stuck
            if solve_puzzle(new_board, visited):
                return True   # Found solution path

            # Backtrack

    return False   # No solution from this state
```

Time Complexity: The total number of states is $4^0 + 4^1 + 4^2 + .. + 4^d = O(4^d)$, so $T(n) = O(4^d)$ where d is the depth of the tree or the number of moves from the initial puzzle state to the goal state.

How to achieve better complexity?

Use an informed search like A* (will be taught in CSE422) that guides the search toward promising states instead of exploring vast numbers of states blindly. A heuristic of A* can be a number of misplaced tiles..

8 Queen Problem

Place 8 queens on a standard chessboard (8×8) such that no two queens attack each other.

Attack Rules:

- A queen can attack another if they are in the **same row**
- A queen can attack another if they are in the **same column**
- A queen can attack another if they are in the **same diagonal**

Objective: Place queens so that no two share a row, column, or diagonal:

Q1							
						Q2	
				Q3			
							Q4
	Q5						
			Q6				
					Q7		
		Q8					

Home Assignment:

Q1) Draw the State Space Tree, Pseudo Code and Time Complexity for 8-Queen Problem.
Q2) Generalize the solution for N-Queen Problem and suggest improvements to the algorithm.